Decision AI Cheat Sheet

The code you need, project by project.

⚡ One call, three question types

```
client.system_one(state, {"name": question})
```

Choice	pick one of N	`r.answers["name"].choice`
Score	a level on a scale	`r.answers["name"].score`
Noul	yes or no, as a probability	`r.answers["name"].noul`

No text,
just answers ☺

Inside this sheet (6 projects)

PAGE 2 **Support Ticket Triage**
SDK, question types, confidence, async, cost

PAGE 3 **Product Review Dashboard**
questions as columns, pandas, composite scores

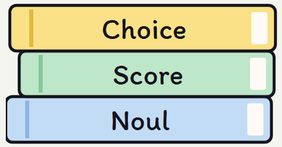
PAGE 4 **Bank Message Router**
accuracy, calibration, thresholds, versions

PAGE 5 **Laya with Ollaya**
Laya on your machine: CLI, API, presets

PAGE 6 **Ollama Decision Endpoint**
nimble in Ollama: raw HTTP, Python client, SDK

PAGE 7 **Jev-Like APIs**
OpenRouter, Liquid, Perplexity, local servers





CHEAT SHEET

Support Ticket Triage

Connect, ask, read, and scale up.

State + questions = answers ☺

Connect and call

Client from `.env`	`load_dotenv(); client = TypeSafeClient()`
Raw HTTP	`httpx2.post(base_url + "/v1/systemone", json=body, headers={"Authorization": "Bearer " + key})`
One call	`client.system_one(state, {"name": question})`
JSON state	`client.system_one({"customer": {...}, "message": "..."}, questions)`

Three question types

Pick one of N	`Choice(instructions=..., criteria={"label": "description"})`
A level on a scale	`Score(instructions=..., criteria=["level 0", "level 1", ...])`
Yes or no	`Noul(instructions=..., criteria={"true": ..., "false": ...})`
See the wire dict	`question.model_dump()`

Read the response

Read answers	`r.answers["name"]`, `r.choices`, `r.scores`, `r.nouls`
Model and tokens	`r.model`, `r.usage.input_tokens`
Typed result	`class T(SystemOneResponse): name: ChoiceAnswer` then `response_model=T`

Confidence, worked out

Choice confidence	`(p_max - 1/n) / (1 - 1/n)`
Score value	`sum(level * p for level, p in probabilities.items())`
Score confidence	`max(0, 1 - spread / even_spread)`, distances from the peak level
Noul on the same scale	`abs(2 * noul - 1)`

Many tickets, safely

Many states at once	`await asyncio.gather(*[aclient.system_one(s, q) for s in states])`
Retries and timeout	`TypeSafeClient(retry=RetryPolicy(max_retries=5), timeout=30.0)`
Catch errors	`except TypeSafeAPIError as e: e.status`
Cost in dollars	`r.usage.input_tokens * 0.042 / 1_000_000`

Remember Confidence is not the top probability. Rescale it by the number of options, then gate on it.

CHEAT SHEET

Product Review Dashboard

Every question becomes a column.

Questions are columns ☺

Questions become columns

A category column	`Choice(instructions, criteria={label: description})` then `.choice`
Its data-quality column	`.confidence`
An ordered number column	`Score(instructions, criteria=[level0, ...])` then `.score`
A probability column	`Noul(instructions)` then `.noul`
All columns in one request	`client.system_one(text, {"col": question, ...})`

Into pandas

Every row at once	`await asyncio.gather(*[aclient.system_one(x, QUESTIONS) for x in texts])`
Answers into pandas	`df["col"] = [r.answers["col"].choice for r in results]`
Counts per group	`pd.crosstab(df["product"], df["aspect"])`
Mean per month	`df.pivot_table(index="month", columns="product", values="sentiment")`

Decide and combine

Yes/no from a probability	`df["refund"] > 0.5`
Rows for a person	`df[df["aspect_conf"] < 0.5], df["refund"].between(0.2, 0.8)`
Score to 0 to 1	`.score / (number of levels - 1)`
Composite score	`w1 * df["a"] + w2 * df["b"] + ...` with weights that sum to 1

Remember One request fills every column for a row. Weights in a composite score sum to 1.

CHEAT SHEET

Bank Message Router

Measure it before you trust it.

Test before launch ☺

Ask every message

Ask every row at once	`await asyncio.gather(*[aclient.system_one(t, {"q": Q}) for t in df.text])`
Prediction column	`df["pred"] = [r.answers["q"].choice for r in results]`
Top probability	`max(r.answers["q"].probabilities.values())`

Score the model

Accuracy	`(df.pred == df.intent).mean()`
Per-class accuracy	`df.groupby("intent").correct.mean()`
Confusion table	`pd.crosstab(df.intent, df.pred)`
Misses	`df[~df.correct][["text", "intent", "pred", "confidence"]]`

Calibration and thresholds

Reliability table	`df.groupby(pd.cut(df.p_top, bins)).correct.agg(["mean", "size"])`
Coverage at a threshold	`(df.confidence >= t).mean()`
Accuracy above a threshold	`df[df.confidence >= t].correct.mean()`
Three bands	`pd.cut(df.confidence, [0, 0.5, 0.9, 1], labels=["human", "confirm", "act"])`

Failure modes and versions

Option order check	`Choice(instructions, criteria=dict(reversed(CRITERIA.items())))`
"None of these"	`criteria=dict(CRITERIA, other="...")`
Version that answered	`r.model`
Pin a version	`TypeSafeClient(model="typesafe/jev-1.13")`

Remember Pick the threshold from coverage and accuracy together, and pin the version you tested.

CHEAT SHEET

Laya with Ollaya

Laya on your own machine.

Port 11435 ☺

Install and manage

Install (Windows)	`irm https://ollaya.dev/install.ps1 | iex`
Install (macOS, Linux)	`curl -fsSL https://ollaya.dev/install.sh | sh`
Start the server	`ollaya serve`
Download a model	`ollaya pull laya`
Models on disk / in memory	`ollaya list` / `ollaya ps`
Model details	`ollaya show laya:en`
Unload	`ollaya stop laya`

Decide in a terminal

One decision in a terminal	`ollaya run laya "text" --preset triage`
All probabilities and timings	`ollaya run laya "text" --preset triage --verbose`
JSON output	`ollaya run laya "text" --preset triage --format json`

TypeSafe SDK, locally

TypeSafe SDK, local	`TypeSafeClient(base_url="http://localhost:11435", api_key="local", model="laya")`
SDK question to plain JSON	`Choice(...).model_dump()`

The native API

Native API	`httpx.post("http://localhost:11435/api/decide", json={"model": "laya", "state": s, "questions": q})`
Routing and timings	`r["routing"]`, `r["total_duration"]`, `r["eval_duration"]` (nanoseconds)
Laya's confidence and act head	`"extras": ["laya"]` then `answer["laya"]`
Force one checkpoint	`"model": "laya:en"` or `"model": "laya:multilingual"`

Presets and custom models

List presets	`ollaya preset list`
Use a preset in the API	`json={"model": "laya", "state": s, "preset": "email"}`
Save a preset	`httpx.post(".../api/presets/create", json={"name": n, "questions": q})`
Model with built-in questions	Modelfile `FROM laya` + `QUESTIONS ./q.json`, then `ollaya create name -f Modelfile`

Remember Ollaya serves on port 11435, so the same TypeSafe SDK code runs locally.

CHEAT SHEET

Ollama Decision Endpoint

A decision model inside Ollama.

Port 11434 ☺

Set up and call

Get a decision model	`ollama pull nimble`
Check it is one	`ollama show nimble` (Capabilities: decision)
Raw HTTP	`httpx.post("http://localhost:11434/v1/systemone", json={"model", "state", "questions"})`
Python client	`ollama.systemone(model="nimble", state=text, questions={...})`
TypeSafe SDK on Ollama	`TypeSafeClient(base_url="http://localhost:11434", api_key="local", model="nimble")`
Hosted Jev	`load_dotenv(); TypeSafeClient()`

Read the answer

Noul	`r.answers["q"].noul`
Choice	`r.answers["q"].choice`, `.probabilities`, `.confidence`
Score	`r.answers["q"].score`, `.legend`, `.probabilities`
Tokens used	`r.usage.input_tokens`

Errors and limits

Catch a rejected request	`except ollama.ResponseError as e: e.status_code, e.error`
Limits	1 to 64 questions, prompt fit per model (`num_ctx`), 64 KB of text and schema, text only

Remember Ollama serves decision models on port 11434. Text only, 1 to 64 questions per request.

CHEAT SHEET

Jev-Like APIs

Same questions, any backend.

Swap the base URL ☺

Hosted, through OpenRouter

Hosted model through OpenRouter	`TypeSafeClient()` with `TYPESAFE_BASE_URL=https://openrouter.ai/api`
Pick a provider's model	`client.system_one(state, questions, model="liquid/d1")`
Exact version that answered	`r.model`
Hosted model IDs on OpenRouter	`jev-latest`, `liquid/d1`, `upstage/solar-decide`, `jaredpalmer/kev-4b`, `perplexity/pplx-decider-v1.2`, `cloudflare/clef`
OpenRouter native endpoint	`POST https://openrouter.ai/api/alpha/decisions` with `provider={...}`

Local and direct backends

Ollaya (Laya)	`TypeSafeClient(base_url="http://localhost:11435", api_key="local"), model="laya"`
Ollama (nimble, tev1)	`TypeSafeClient(base_url="http://localhost:11434", api_key="local"), model="nimble"`
llama.cpp `llama-server`	`TypeSafeClient(base_url="http://localhost:9931", api_key="local")`
Liquid direct	`TypeSafeClient(base_url="https://api.liquid.ai", api_key=os.environ["LIQUID_API_KEY"]), model="d1"`
Perplexity direct (own path)	`POST https://api.perplexity.ai/v1/decisions, model="pplx-decider-v1.2"`

Compare the answers

Agreement across models	`preds.nunique(axis=1)`
Majority answer	`preds.mode(axis=1)[0]`

Remember — Only the base URL and the model ID change. The questions stay the same.